

FSOC COARSE TRACKER

AI-Based Virtual Camera Tracking System for Coarse
Alignment of Mobile Free Space Optical Communication
(FSOC) Terminals

User & Developer Manual · Version 0.1.0 · September 2026

Table of Contents

1. Introduction	4
1.1 Problem context	4
1.2 What the system does	4
1.3 Key capabilities	4
2. Getting started	5
2.1 Prerequisites	5
2.2 Installation	5
2.3 Sanity check	5
2.4 First interactive run	5
2.5 Interactive controls	6
3. Operator guide	7
3.1 Operation modes	7
3.2 CLI options reference	7
3.3 Example runs	9
3.4 Understanding the HUD	9
4. Outputs and metrics	11
4.1 Metrics definitions	11
4.2 CSV log format	11
4.3 Video recording	12
4.4 Selftest report	12
5. Architecture	13
5.1 Overall data flow	13
5.2 Module reference	13
5.3 Coordinate and projection model	14
5.4 Detection and tracking	14
5.5 Coarse-alignment controller	15
5.6 Disturbance models	15
5.7 Metrics implementation	15
6. Tuning and performance	17
6.1 PID gains and feedforward	17
6.2 Acquisition / scan	17
6.3 Disturbance stress levels	17
6.4 Rendering and FPS	17
6.5 Regression testing with selftest	17
7. SIH requirements traceability matrix	18
8. Troubleshooting	19
9. Extension guide	20
9.1 New beacon trajectories	20

- 9.2 Learned (AI) detector 20
- 9.3 Multi-beacon support 20
- 9.4 Data export for analytics 20
- 10. Appendix 21**
 - 10.1 Configuration reference (fscam/config.py) 21
 - 10.2 File layout 22
 - 10.3 Quick CLI table 23

1. Introduction

1.1 Problem context

Free Space Optical Communication (FSOC) offers gigabit-to-terabit data rates for next-generation mobile networks, but deploying FSOC links between mobile platforms (satellites, UAVs, ground stations) requires **Pointing, Acquisition and Tracking (PAT)** of very narrow laser beams. PAT happens in two stages:

1. **Coarse alignment** - the transmitting terminal must locate the remote terminal and keep it inside its camera Field-of-View (FOV), and
2. **Fine alignment** - fine pointing optimisation of the beam.

Developing and testing these algorithms on real hardware is expensive. This project is the SIH 2026 submission for ISRO Problem Statement **26169**: "*Development of an AI-Based Virtual Camera Tracking System for Coarse Alignment of Mobile Free Space Optical Communication (FSOC) Terminals*" - a purely software platform that simulates the coarse-alignment loop using a *virtual* camera, a *virtual* scene, and moving optical beacons.

1.2 What the system does

The system renders a synthetic night-sky scene containing a moving optical beacon, projects it through a virtual pan/tilt camera with configurable FOV, then runs the full coarse-alignment loop:

observe → detect the beacon → estimate its position → steer the virtual gimbal to keep it centred.

The same detection/tracking pipeline runs against a **real USB webcam**, so the algorithm developed here can be validated on both simulated and physical cues without special hardware.

1.3 Key capabilities

- Configurable virtual environment (scene, camera, beacon, trajectory)
- Moving beacons with selectable motion (lissajous, circle, line, stationary)
- Movable virtual camera with pan/tilt and FOV control (manual or automatic)
- Automatic acquisition - beacon is located without manual pointing (raster scan)
- Continuous tracking with Kalman filtering and a PID + velocity-feedforward controller
- Disturbance simulation (sensor noise, atmospheric turbulence, gimbal vibration, blur)
- Real-time statistics (pointing error, lock state, detection rate, FPS, latency)
- Webcam mode for physical-beacon experiments
- Video recording and CSV metrics export for post-analysis and BI tools (e.g. Tableau)

2. Getting started

2.1 Prerequisites

Requirement	Version	Notes
Python	3.10+	Developed and tested on Python 3.14
numpy	≥ 1.26	Numerical arrays / rendering
opencv-python	≥ 4.9	Image processing, windows, Kalman filter
Display (interactive)	any X11 / Wayland	Only needed for the GUI window; fully headless otherwise

2.2 Installation

```
cd ~/Projects/fsoc-tracker
python -m venv venv          # if not already created
source venv/bin/activate
pip install -r requirements-fscam.txt
```

2.3 Sanity check

Run the headless regression test. It renders 320 frames, verifies that coarse alignment locks the beacon, prints a metrics summary, and returns exit code 0/1.

```
python run_virtual_camera.py --selftest --frames 900
```

Expected tail of output (values vary with seed):

```
frames: 900
fps: 97.5
vision_ms: 0.55
frame_ms: 10.41
detection_rate_pct: 75.7
locked: True
mean_err_deg: 0.588
peak_err_deg: 17.948
last_err_deg: 0.322
SELFTEST PASS
```

2.4 First interactive run

```
python run_virtual_camera.py
```

A window opens showing the simulated night sky. A reddish beacon appears in the scene; the system first **SCANNING** (orange reticle, sweeping the gimbal), then acquires the beacon and centres it. The HUD turns green and reports **LOCKED** once the pointing error stays below the lock threshold long enough.

2.5 Interactive controls

Key	Action
a	Toggle automatic coarse alignment (on/off)
d	Toggle disturbance simulation (on/off)
p	Pause / resume simulation time
h	Toggle the HUD overlay
r	Recentre camera and re-initiate acquisition (reset)
Esc / q	Quit
Arrow keys	Manual pan/tilt (only while autotrack is off)

3. Operator guide

3.1 Operation modes

Mode	Purpose	Ground truth
<code>virtual</code> (default)	Full simulation: synthetic scene, beacon trajectory, all disturbances	Available (true beacon position is known), enabling exact pointing-error measurement and lock logic
<code>webcam</code>	Physical experiment: detect and track a bright optical beacon (LED/light source) seen by a USB camera	Unavailable; lock falls back to pixel (off-boresight) error

```
python run_virtual_camera.py --mode virtual
python run_virtual_camera.py --mode webcam --camera 0 --width 640 --height 480
```

In webcam mode the trajectory, vibration and pixel-turbulence models do not apply; sensor noise and blur may still be enabled to stress the detector.

3.2 CLI options reference

3.2.1 General

Option	Default	Description
<code>--mode {virtual,webcam}</code>	<code>virtual</code>	Frame source
<code>--width</code>	<code>800</code>	Viewport width (px)
<code>--height</code>	<code>600</code>	Viewport height (px)
<code>--fov</code>	<code>20.0</code>	Horizontal FOV (deg)
<code>--pan</code>	<code>0.0</code>	Initial pan (deg)
<code>--tilt</code>	<code>6.0</code>	Initial tilt (deg)
<code>--seed</code>	<code>42</code>	RNG seed (reproducible scenes)
<code>--headless</code>	off	Run without a window (useful for automation)
<code>--hud</code> / <code>--no-hud</code>	on	Toggle HUD overlay

3.2.2 Beacon trajectory

Option	Default	Description
<code>--trajectory {lissajous,circle,line,stationary}</code>	<code>lissajous</code>	Beacon motion model
<code>--traj-center-az</code>	<code>18.0</code>	Trajectory centre, azimuth (deg)

Option	Default	Description
<code>--traj-center-el</code>	8.0	Trajectory centre, elevation (deg)
<code>--traj-amp-az</code>	7.0	Azimuth amplitude (deg)
<code>--traj-amp-el</code>	5.0	Elevation amplitude (deg)
<code>--traj-freq-az</code>	0.05	Azimuth oscillation frequency (Hz)
<code>--traj-freq-el</code>	0.033	Elevation oscillation frequency (Hz)
<code>--traj-speed</code>	1.0	Time-scale multiplier

3.2.3 Disturbances

Option	Default	Description
<code>--disturbances</code> / <code>--no-disturbances</code>	on	Master switch for disturbance simulation
<code>--noise</code>	0.02	Sensor noise sigma (fraction of 255)
<code>--turbulence</code>	2.5	Atmospheric turbulence: pixel wander magnitude
<code>--blur</code>	3	Sensor blur kernel size (0 = off)
<code>--jitter</code>	0.05	Gimbal vibration amplitude (deg)

3.2.4 Coarse-alignment control

Option	Default	Description
<code>--autotrack</code> / <code>--no-autotrack</code>	on	Enable PID tracking
<code>--kp</code>	4.0	Proportional gain
<code>--ki</code>	0.03	Integral gain
<code>--kd</code>	0.6	Derivative gain
<code>--max-rate</code>	6.0	Maximum gimbal slew rate (deg/s)
<code>--scan-window</code>	34.0	Acquisition raster-scan width (deg)
<code>--lock-threshold</code>	0.6	Pointing-error threshold for lock (deg)
<code>--lock-frames</code>	15	Consecutive frames below threshold to declare lock
<code>--max-lost</code>	20	Frames without detection before re-acquisition

3.2.5 Scene and output

Option	Default	Description
<code>--horizon</code>	-14.0	Horizon elevation in the scene (deg)
<code>--sun</code>	off	Render a bright sun disk in the scene
<code>--stars</code>	1400	Number of background stars

Option	Default	Description
<code>--record</code>	(none)	Record processed video (<code>.mp4</code> or <code>.avi</code>)
<code>--log</code>	(none)	Append per-frame metrics to CSV
<code>--camera</code>	<code>0</code>	Webcam index (webcam mode)
<code>--selftest</code>	off	Run headless regression test and exit
<code>--frames</code>	<code>320</code>	Number of frames to run for <code>--selftest</code>

3.3 Example runs

Acquire and track the default moving beacon:

```
python run_virtual_camera.py --trajectory lissajous
```

Stress test with heavy disturbances:

```
python run_virtual_camera.py --noise 0.08 --turbulence 8 --jitter 0.3
```

Track a slow circular target with a brighter, wider FOV:

```
python run_virtual_camera.py --trajectory circle --traj-freq-az 0.03 --fov 30 \
  --traj-amp-az 4 --traj-amp-el 4
```

Live experiment with a USB camera (point an LED at the lens):

```
python run_virtual_camera.py --mode webcam --camera 0 --record demo.mp4 --log data/
manual_run.csv
```

Open-loop demonstration (no autotrack; control the gimbal yourself with arrow keys):

```
python run_virtual_camera.py --no-autotrack
```

3.4 Understanding the HUD

HUD element	Meaning
State (SCANNING / TRACKING / LOCKED / MANUAL)	Acquisition & lock state of the controller
FPS Frame ms Vision ms	Loop performance: frames/s, full-loop time, perception latency
Detections %	Fraction of frames where the detector found the beacon
PAN / TILT	Current commanded gimbal angles
FOV	Horizontal field of view
Pointing err deg	True angular error between boresight and beacon (virtual mode)

HUD element	Meaning
Measured off-boresight deg	Angular error computed from the detected centroid (both modes)
Green sparkline	Rolling history of the pointing error
Cross-hair moving reticle	Filtered beacon position (green = locked, yellow = tracking)
Orange reticle + ACQUIRING...	Raster-scan search in progress

4. Outputs and metrics

4.1 Metrics definitions

Metric	Definition
Pointing error (<code>err_deg</code>)	<code>angular_sep(pan, tilt, beacon_az, beacon_el)</code> - true geodesic angle between commanded boresight and the real beacon direction. Only available in virtual mode (ground truth is known).
Measured error (<code>meas_deg</code>)	Angular offset of the Kalman-filtered detection centroid from the image centre: <code>atan2(distance_px, focal_px)</code> . Available in both modes.
Lock	<code>err_deg</code> (virtual) or <code>meas_deg</code> (webcam) below <code>--lock-threshold</code> for <code>--lock-frames</code> consecutive frames.
Detection rate	Fraction of frames with a raw blob detection.
FPS / frame time	Exponential moving average of loop frequency and duration.
Vision latency	EMA of detector runtime (ms) - a proxy for perception pipeline latency.

4.2 CSV log format

Produced by `--log path.csv`; one row per frame:

```
t,pan,tilt,err_deg,meas_deg,state,det,fps
```

Column	Type	Description
<code>t</code>	float	Simulation time (s)
<code>pan</code> , <code>tilt</code>	float	Commanded gimbal angles (deg)
<code>err_deg</code>	float (blank if unavailable)	True pointing error (deg)
<code>meas_deg</code>	float (blank if blank)	Measured off-boresight error (deg)
<code>state</code>	str	<code>SEARCH</code> , <code>TRACK</code> (or <code>LOST</code>)
<code>det</code>	int 0/1	Whether a raw detection existed this frame
<code>fps</code>	float	Loop rate for this frame

Example:

```
t,pan,tilt,err_deg,meas_deg,state,det,fps
0.016,0.064,6.0,17.9485,,SEARCH,0,177.1
0.032,0.128,6.0,17.9216,,SEARCH,0,168.2
...
3.504,13.59,6.80,0.412,0.419,TRACK,1,96.3
```

4.3 Video recording

`--record out.mp4` (or `.avi`) writes the processed frames (HUD included) at 60 fps.

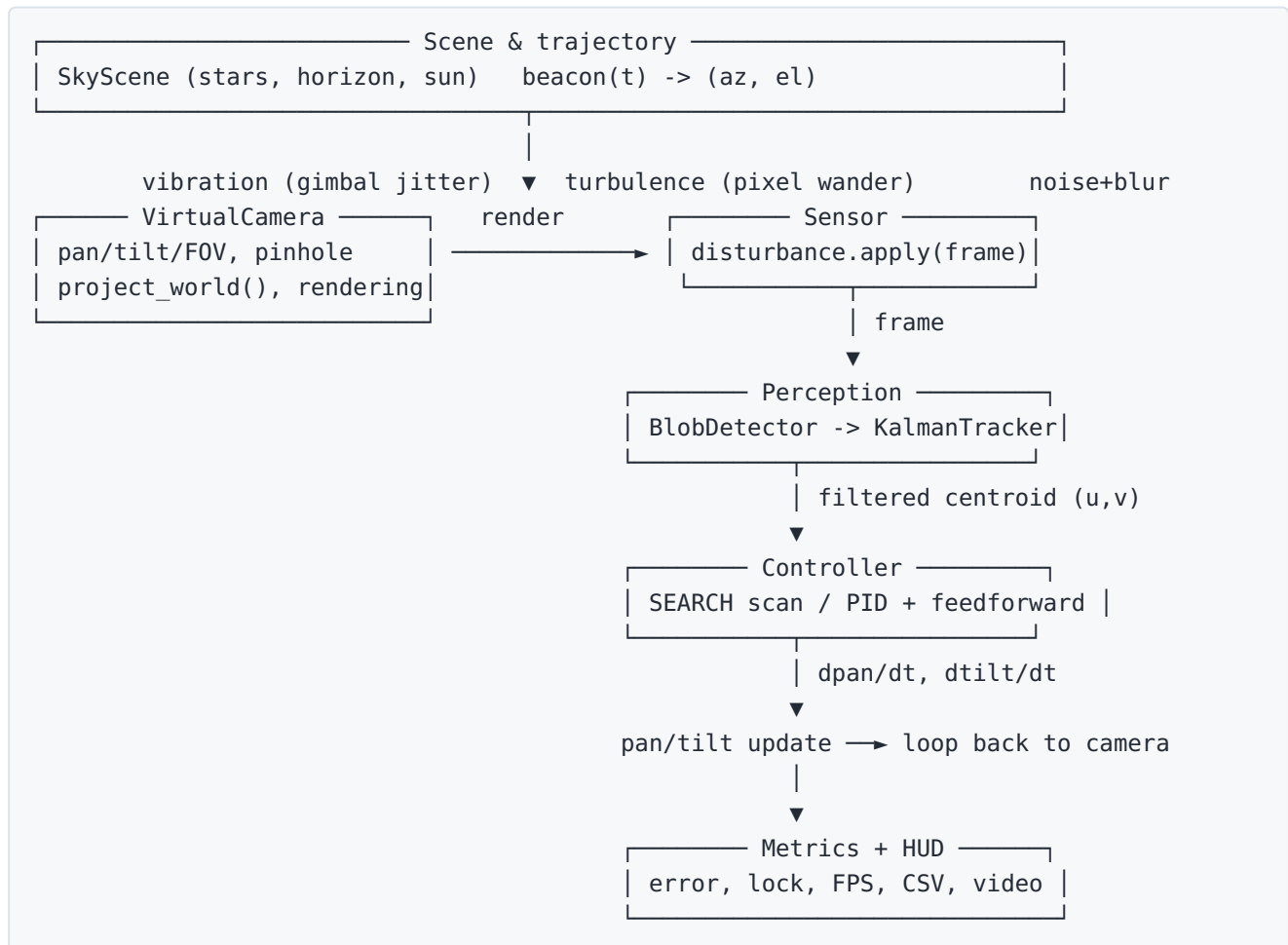
4.4 Selftest report

`--selftest` prints a machine-readable summary used by CI/regression checks:

Key	Description
<code>frames</code>	Frames simulated
<code>fps</code> , <code>vision_ms</code> , <code>frame_ms</code>	Performance
<code>detection_rate_pct</code>	Detector success rate
<code>locked</code>	Whether the coarse-alignment loop reached lock
<code>mean_err_deg</code>	Mean true pointing error over the run
<code>peak_err_deg</code>	Worst-case pointing error
<code>last_err_deg</code>	Final pointing error

5. Architecture

5.1 Overall data flow



In `webcam` mode the left half (renderer/simulation) is replaced by a real camera frame; the perception, controller and metrics pipeline is identical.

5.2 Module reference

Module	File	Responsibility
<code>Config</code>	<code>fscam/config.py</code>	Single dataclass with every tunable parameter
<code>SkyScene</code>	<code>fscam/scene.py</code>	Star field, horizon, optional sun, beacon trajectory factory
<code>VirtualCamera</code>	<code>fscam/camera_model.py</code>	Pinhole projection, pan/tilt pose, FOV, pixel ↔ angle conversions
<code>renderer</code>	<code>fscam/renderer.py</code>	Draws the scene into the camera frame (background, stars, beacon bloom)
<code>Disturbances</code>	<code>fscam/disturbances.py</code>	

Module	File	Responsibility
		Vibration (OU), turbulence (OU), sensor noise, blur
BlobDetector / KalmanTracker	fscam/vision.py	Bright-blob perception; constant-velocity Kalman filtering with gating
CoarseController	fscam/controller.py	SEARCH/TRACK/LOST state machine; raster scan; PID + feedforward
Metrics	fscam/metrics.py	Error histories, lock detection, FPS/latency EMAs, summaries
ui	fscam/ui.py	HUD overlay, reticles, sparkline
VirtualSource / WebcamSource	fscam/sources.py	Frame producers (simulation or camera)
app	fscam/app.py	Main loop, CLI parsing, keyboard handling, recording, CSV
entry point	run_virtual_camera.py	Thin launcher for fscam.app.main

5.3 Coordinate and projection model

The camera poses uses a right-handed frame with axes:

- **forward** `w` = direction of boresight at (pan, tilt)
- **right** `r` (horizontal)
- **up** `u` (perpendicular)

A target direction `d` is expressed in camera coordinates as:

```
az_c = atan2(d . r, d . w)    # positive = right of centre
el_c = atan2(d . u, d . w)    # positive = above centre
```

Pixels follow a gnomonic (pinhole) projection using the focal length derived from the horizontal FOV:

```
f_px = (W / 2) / tan(FOV_h / 2)
u     = W / 2 + f_px * tan(az_c)
v     = H / 2 - f_px * tan(el_c)
```

`WorldCamera.pixel_to_angles()` inverts this to map a detection back to the angular domain used by the controller, which makes the control law resolution-independent.

5.4 Detection and tracking

BlobDetector thresholds the grayscale image relative to its brightest pixels, cleanly separates components, keeps compact bright blobs (area between 4 and 900 px, minimum mean score), and returns an **intensity-weighted subpixel centroid** for the best candidate.

KalmanTracker implements a constant-velocity Kalman filter over `(u, v, du, dv)`:

- predicts each frame (bridges the inter-frame gap and provides graceful occlusion handling),
- corrects detections inside a Mahalanobis-like pixel gate (rejects outliers),
- increments a `lost` counter on missing detections and returns `None` after `--max-lost` frames so the controller can re-acquire.

Gating plus the intensity-weighted centroid keeps the filter stable against star-like distractors.

5.5 Coarse-alignment controller

A state machine drives the virtual gimbal:

- **SEARCH (acquisition):** raster-scan pan/tilt across `--scan-window` until a beacon is detected. This fulfils the "locate the remote terminal without manual pointing" requirement.
- **TRACK:** PID on the angular error plus a **velocity feedforward** term. The feedforward estimates beacon angular velocity from the filtered centroid history, which removes most of the steady-state lag. The combined command is clamped to `--max-rate`.
- **LOST:** after `--max-lost` frames without a filtered detection, returns to SEARCH.

The feedforward is important: with proportional control alone the steady-state error tends toward `target_slew / Kp` (e.g. a beacon slewing $4.8^\circ/\text{s}$ against $Kp=3$ leaves about 1.6° of bias).

5.6 Disturbance models

Disturbance	Model	Config
Gimbal vibration	Ornstein-Uhlenbeck process added to camera pan/tilt at render time	<code>--jitter</code>
Atmospheric turbulence	OU process shifting the beacon pixel centre, plus a Gaussian blur at the sensor	<code>--turbulence</code> , <code>--blur</code>
Sensor noise	Zero-mean Gaussian with std = <code>--noise</code> x 255 added to the frame	<code>--noise</code>

Enabling/disabling all of them at runtime with the `d` key lets a judge observe tracking degraded and recovered live.

5.7 Metrics implementation

`Metrics` maintains rolling dequeues of the true and measured errors (for the sparkline and CSV), a peak tracker, a lock frame counter, and EMA performance timers. Pointing error uses

the true scene beacon azimuth/elevation against the *commanded* gimbal pose, decoupled from detection so that controller quality and perception quality are reported separately.

6. Tuning and performance

6.1 PID gains and feedforward

- `--kp` dominates residual error; aggressive values can oscillate near target turn-around.
- `--kd` dampens overshoot on fast changes; keep small (0.5-1.0).
- `--ki` removes small long-term bias; keep small to avoid wind-up (`--max-rate` limits slew).
- The feedforward term is tuned by `_ff_alpha` in `fscam/controller.py`. Raise it for faster slew-accurate tracking, lower it for smoother estimates.
- `--max-rate` should exceed the peak beacon angular rate, or tracking will saturate.

6.2 Acquisition / scan

- `--scan-window` sets the uncertainty region to sweep during acquisition. It should cover the expected beacon region (the wide default of 34° comfortably spans the default trajectory).
- `--scan-speed` and `--scan-row-step` (code defaults 4°/s, 2.5°) trade acquisition time against sweep coverage.

6.3 Disturbance stress levels

<code>--noise</code>	<code>--turbulence</code>	<code>--jitter</code>	Effect
0.02	2.5	0.05	Nominal (default)
0.05	8	0.3	Pronounced scintillation & vibration; tracking still holds
0.10+	12+	0.5+	Near-visibility limit; demonstrates robust filter behaviour

6.4 Rendering and FPS

- Default 800x600 renders comfortably above 90 fps on a desktop.
- Lower `--width` / `--height` and `--stars` for weak hardware.
- The `--selftest` path uses 400x300 for fast regression runs.

6.5 Regression testing with selftest

`--selftest` is deterministic for a fixed `--seed`. Use it as the automated gate (CI) after any code change: it asserts the loop can lock and that mean error stays low.

7. SIH requirements traceability matrix

ISRO functional requirement	Implementation	How to demonstrate
Configurable virtual environment	<code>Config</code> , <code>scene.py</code> , <code>renderer.py</code> , <code>camera_model.py</code>	Change scene/target/camera via CLI flags and reload
Moving targets (one or more beacons)	<code>make_trajectory</code> in <code>scene.py</code> (lissajous/circle/line/stationary)	Run <code>--trajectory circle</code> and watch the beacon sweep
Movable virtual camera	pan/tilt in <code>camera_model.py</code> , <code>controller.py</code> , arrow keys	Toggle <code>--no-autotrack</code> and steer with arrow keys
Automatic detection	<code>BlobDetector</code> + SEARCH scan in <code>controller.py</code>	Start off-target; observe <code>ACQUIRING...</code> until found
Continuous tracking	<code>KalmanTracker</code> + PID/feedforward	Confirm HUD reads <code>LOCKED</code> while the beacon moves
Disturbances (noise, turbulence, vibration, camera movement)	<code>disturbances.py</code>	Press <code>d</code> ; watch error rise and recover when re-enabled
Real-time statistics	<code>metrics.py</code> , <code>ui.py</code>	Read error/lock/FPS/latency on the HUD and in the CSV log
Software-only, no special hardware	whole project (virtual + webcam modes)	Run anywhere with Python + OpenCV

8. Troubleshooting

Symptom	Cause / fix
<code>RuntimeError: Cannot open camera index 0</code>	The webcam is busy or absent. Try <code>--camera 1</code> ; list devices with <code>ls /dev/video*</code> .
GUI does not open in interactive mode	Run under a desktop session, or use <code>--headless</code> / <code>--selftest</code> (fully deterministic).
<code>QFontDatabase: Cannot find font directory ... cv2/qt/fonts</code>	Cosmetic OpenCV-Qt warning; safe to ignore.
Slow frame rate	Reduce <code>--width/--height</code> and <code>--stars</code> ; disable <code>--no-hud</code> while developing.
<code>SELFTEST FAIL</code>	Regression in tracking; check <code>mean_err_deg</code> , <code>fps</code> , and recent module changes. The test is deterministic for a fixed seed.
Beacon never acquired	<code>--scan-window</code> too small for the trajectory region; increase it or move <code>--traj-center-*</code> .
Tracking lags the target	Increase <code>--max-rate</code> above beacon slew, or raise feedforward <code>_ff_alpha</code> in <code>controller.py</code> .

9. Extension guide

9.1 New beacon trajectories

`scene.py: make_trajectory(cfg)` returns `traj(t) -> (az, el)`. Add a branch in the `kind` dispatch for a new name and it becomes available via `--trajectory`. Trajectories are pure functions of time, so any parametrisation is supported.

9.2 Learned (AI) detector

The perception stage (`vision.BlobDetector.detect`) is a small, drop-in interface:

```
detect(frame) -> (u, v, score, area) | None
```

Swap the classical threshold+blob logic for a learned model (e.g. YOLO, a small CNN trained on rendered frames, or a transformer tracker) while keeping `KalmanTracker`, the controller and the metrics untouched. This is the natural path to strengthening the "AI-based" claim beyond the statistical baseline.

9.3 Multi-beacon support

- Renderer: extend `SkyScene` to hold a list of beacons.
- Perception: return a list of candidates from `BlobDetector` and run one `KalmanTracker` per ID.
- Controller/metrics: extend the lock logic per beacon, and add a beacon-ID column to the CSV.

9.4 Data export for analytics

The CSV (`--log`) is directly consumable by Tableau, Excel, or pandas. Suggested Tableau dashboard: - Line chart of `err_deg` and `meas_deg` over `t`, partitioned by `state` - Histogram of `err_deg` while `state=TRACK` to show steady-state distribution - Scatter `pan` vs `tilt` to visualise the gimbal path during acquisition - KPI tiles for lock duration, detection rate, mean/peak error

See Section 10 for a full field reference.

10. Appendix

10.1 Configuration reference (`fscam/config.py`)

Field	Default	CLI flag	Description
<code>mode</code>	<code>virtual</code>	<code>--mode</code>	Frame source
<code>width</code> / <code>height</code>	<code>800</code> / <code>600</code>	<code>--width</code> / <code>--height</code>	Viewport size (px)
<code>fov_h_deg</code>	<code>20.0</code>	<code>--fov</code>	Horizontal FOV (deg)
<code>pan_deg</code> / <code>tilt_deg</code>	<code>0.0</code> / <code>6.0</code>	<code>--pan</code> / <code>--tilt</code>	Initial pose
<code>pan_limits</code> / <code>tilt_limits</code>	<code>(-90,90)</code> / <code>(-30,30)</code>	(config-only)	Gimbal travel limits
<code>trajectory</code>	<code>lissajous</code>	<code>--trajectory</code>	Beacon motion
<code>traj_center_az</code> / <code>traj_center_el</code>	<code>18.0</code> / <code>8.0</code>	<code>--traj-center-*</code>	Trajectory centre
<code>traj_amp_az</code> / <code>traj_amp_el</code>	<code>7.0</code> / <code>5.0</code>	<code>--traj-amp-*</code>	Amplitudes
<code>traj_freq_az</code> / <code>traj_freq_el</code>	<code>0.05</code> / <code>0.033</code>	<code>--traj-freq-*</code>	Frequencies (Hz)
<code>traj_speed</code>	<code>1.0</code>	<code>--traj-speed</code>	Time multiplier
<code>horizon_deg</code>	<code>-14.0</code>	<code>--horizon</code>	Scene horizon
<code>num_stars</code>	<code>1400</code>	<code>--stars</code>	Star-field size
<code>seed</code>	<code>42</code>	<code>--seed</code>	RNG seed
<code>beacon_sigma_px</code>	<code>1.8</code>	(config-only)	Beacon point-spread radius
<code>sun</code>	<code>False</code>	<code>--sun</code>	Render sun disk
<code>noise</code>	<code>0.02</code>	<code>--noise</code>	Sensor noise sigma
<code>turbulence_px</code>	<code>2.5</code>	<code>--turbulence</code>	Turbulence wander (px)
<code>blur</code>	<code>3</code>	<code>--blur</code>	Blur kernel (0=off)
<code>jitter_deg</code>	<code>0.05</code>	<code>--jitter</code>	Gimbal vibration (deg)
<code>kp</code> / <code>ki</code> / <code>kd</code>	<code>4.0</code> / <code>0.03</code> / <code>0.6</code>	<code>--kp/--ki/--kd</code>	PID gains
<code>max_rate_deg_s</code>	<code>6.0</code>	<code>--max-rate</code>	Slew limit (deg/s)
<code>scan_window_deg</code>	<code>34.0</code>	<code>--scan-window</code>	Acquisition scan width

Field	Default	CLI flag	Description
scan_speed_deg_s	4.0	(config-only)	Scan sweep rate
scan_row_step_deg	2.5	(config-only)	Scan raster pitch
lock_threshold_deg	0.6	--lock-threshold	Lock error band
lock_frames	15	--lock-frames	Frames for lock
max_lost	20	--max-lost	Frames before re-acquisition
autotrack	True	--autotrack/--no-autotrack	Enable PID tracking
disturbances	True	--disturbances/--no-disturbances	Master disturbance switch
show_hud	True	--hud/--no-hud	HUD visibility
record	""	--record	Output video path
log	""	--log	Output CSV path
camera_index	0	--camera	Webcam index
headless	False	--headless	No-window mode
sim_dt	0.016	(config-only)	Simulation step (s)

10.2 File layout

```

fsoc-tracker/
├── fscam/
│   ├── __init__.py
│   ├── app.py           # CLI, main loop, key handling, recording/CSV
│   ├── camera_model.py  # pinhole projection, pan/tilt, FOV
│   ├── config.py        # Config dataclass
│   ├── controller.py    # SEARCH/TRACK/LOST, PID + feedforward
│   ├── disturbances.py  # vibration, turbulence, noise, blur
│   ├── metrics.py       # errors, lock, FPS, latency
│   ├── renderer.py      # scene -> frame rendering
│   ├── scene.py         # stars, trajectories, SkyScene
│   ├── sources.py       # Virtual/Webcam frame sources
│   ├── ui.py            # HUD overlay
│   └── vision.py        # BlobDetector, KalmanTracker
├── docs/
│   ├── manual.md        # this manual
│   ├── style.css        # PDF print stylesheet
│   ├── build_pdf.sh     # markdown -> PDF builder
│   └── manual.pdf       # generated PDF
├── run_virtual_camera.py # launcher
├── requirements-fscam.txt
└── venv/                # Python environment

```

10.3 Quick CLI table

```
python run_virtual_camera.py           # interactive virtual demo
python run_virtual_camera.py --mode webcam      # USB webcam tracking
python run_virtual_camera.py --selftest --frames 900 # regression check (exit code)
python run_virtual_camera.py --trajectory circle  # different beacon motion
python run_virtual_camera.py --noise 0.08 --turbulence 8 --jitter 0.3 # stress test
python run_virtual_camera.py --record demo.mp4 --log data.csv          # capture
python run_virtual_camera.py --help           # full option help
```